Mini Project Report

On

AI-Powered Teacher-Facing Student Information Assistant

Submitted by:

Prasad Bhalerao (RBT24CB026)

Under the guidance of

Prof. Sumedha Raut

Course: Artificial Intelligence

Department of Computer Science and Business Systems

Academic Year: 2026–27 (Semester VI)

Table of Contents

1. Abstract 3
2. Introduction 4
2.1 Background and Motivation 4
2.2 Problem Definition 4
2.3 Objectives 4
3. Literature Survey / Existing System 5
3.1 Review of Related Tools and Approaches 5
3.2 Gaps Identified 5
4. Proposed System 6
4.1 System Overview 6
4.2 Features and Scope 6
4.3 System Architecture 6
4.4 Database Design 7
5. Methodology / Implementation 9
5.1 Tools and Technologies Used 9
5.2 Synthetic Dataset Generation 9
5.3 Authorisation Model 10
5.4 Exact Attendance Arithmetic 11
5.5 AI Orchestration 11
5.6 Streaming and Credential Handling 12
6. Results and Discussion 14
6.1 Outputs Achieved 14
6.2 Verification and Test Coverage 14
6.3 A Defect Found by Probing the Deployed Assistant 14
6.4 Authorisation Verification 16
6.5 Comparison with Existing Approaches 16
7. Conclusion and Future Scope 17
7.1 Summary of Achievements 17
7.2 Limitations 17
7.3 Future Scope 17
7.4 Reflection 18
8. References 19
Project Repository and Live Deployment 19

1. Abstract

This project implements an AI-powered student information assistant for teaching staff in a third-year Computer Science and Business Systems department. Faculty members sign in, view only the subjects assigned to them, and inspect attendance, academic performance, assignment submissions and extracurricular participation, either through dedicated application pages or by asking questions in plain English.

The system is a monorepo containing a Next.js frontend and a FastAPI backend over a PostgreSQL database holding 27,975 synthetic records across eleven normalised tables. The distinguishing design decision is that the language model is never given database access. Instead it may call thirteen fixed, access-controlled functions, each of which re-verifies the signed-in teacher's permissions against the database before reading anything. No SQL is generated by the model, and a request for another teacher's subject is refused in code rather than by instruction.

All attendance arithmetic uses exact rational numbers so that a student at precisely the threshold is never misclassified by floating-point rounding. The system distinguishes unevaluated work from a recorded zero, and genuinely overdue submissions from work awaiting marking. Responses stream token by token from the chosen model provider. The application is deployed and verified end to end, with 208 automated tests covering dataset integrity, business logic, authorisation and the AI layer.

2. Introduction

2.1 Background and Motivation

Departmental student data is usually spread across spreadsheets, attendance registers and examination records. A teacher who wants to answer a simple question — which students in my class are at risk of falling below the attendance requirement — typically has to open several files, filter them by hand and compute percentages manually. The work is repetitive, easy to get wrong, and has to be redone every time the question changes slightly.

Large language models appear to offer a solution: ask the question in English and receive an answer. In practice, connecting a language model directly to institutional records introduces two serious problems. First, a model asked for a number it cannot retrieve will often produce a plausible-looking figure instead of admitting the gap. Second, a model given broad database access has no inherent notion of which teacher may see which student, so a persuasive instruction in a chat message could expose records the user is not entitled to.

This project treats both problems as architectural rather than conversational. Every number shown to a teacher is computed in Python from database rows; the model's role is to interpret the question and explain the result, never to calculate it. Access control is enforced in backend code and re-checked inside every function the model can invoke, so the system prompt is a guide to behaviour rather than a security boundary.

2.2 Problem Definition

Design and implement a teacher-facing system that lets faculty query synthetic student information in natural language, such that: every factual answer derives from database records and verified calculations rather than model inference; a teacher can access only the subjects assigned to them and only students enrolled in those subjects; and the restriction holds even when the request is made through the AI assistant or by editing a URL directly.

2.3 Objectives

- Generate a reproducible synthetic dataset of 250 students across 12 subjects.
- Model the data in a normalised relational schema with integrity constraints enforced by the database.
- Implement server-side authorisation resolving each teacher's scope from the database on every request.
- Expose the data to a language model through fixed, access-controlled functions, not generated SQL.
- Compute all attendance and performance figures using exact arithmetic in a single shared module.
- Stream model responses progressively and report provider failures honestly instead of substituting invented text.
- Support multiple AI providers so a user can supply their own API key without it being stored.
- Validate the system with automated tests covering data, logic, authorisation and AI behaviour.

3. Literature Survey / Existing System

3.1 Review of Related Tools and Approaches

Existing approaches to querying institutional data with language models fall into three broad categories, each with characteristic limitations relevant to this problem.

Approach	How it works	Limitation for this problem
Conventional ERP / LMS dashboards	Fixed reports and filters built by developers	Any question the designer did not anticipate requires a code change or manual spreadsheet work
Text-to-SQL systems	The model writes SQL against a schema description, which is then executed	Query correctness depends on model output; access control must be expressed in prompt text, which can be argued with
Retrieval-Augmented Generation (RAG)	Records are embedded as vectors and semantically similar ones retrieved as context	Similarity search cannot express 'only this teacher's subjects', and approximate retrieval is unsuitable for exact counts
Function or tool calling	The model selects from a fixed set of developer-written functions	The set of possible operations is finite and reviewable; each function can enforce permissions in code

Table 3.1: Comparison of approaches for natural-language access to relational records

3.2 Gaps Identified

Three gaps emerged from this review and directly shaped the design of the proposed system.

- Authorisation expressed in prompts is not authorisation. Instructing a model to respect permissions leaves the restriction subject to persuasion. Permission checks must execute in code on the server, independently of what the model decides to do.
- Approximate retrieval is wrong for exact questions. Attendance percentages, counts of students below a threshold and marks averages have single correct answers. Vector similarity is the wrong instrument; indexed relational queries with exact arithmetic are the right one.
- A truncated result set invites a false total. If a function returns the ten worst cases without stating how many exist, a model will reasonably but incorrectly report ten as the total. Every function must return explicit counts separate from the rows it displays. This gap was confirmed experimentally during development, as described in Section 6.3.

4. Proposed System

4.1 System Overview

The proposed system is a web application in which a faculty member signs in, is issued a signed session, and thereafter sees only data within their teaching scope. A browser-based frontend renders nine pages of analytics and a chat interface. A backend resolves permissions, performs all calculations and orchestrates the language model. A relational database holds the records. An external model provider supplies language understanding only; it receives the question and authorised results, never direct database access.

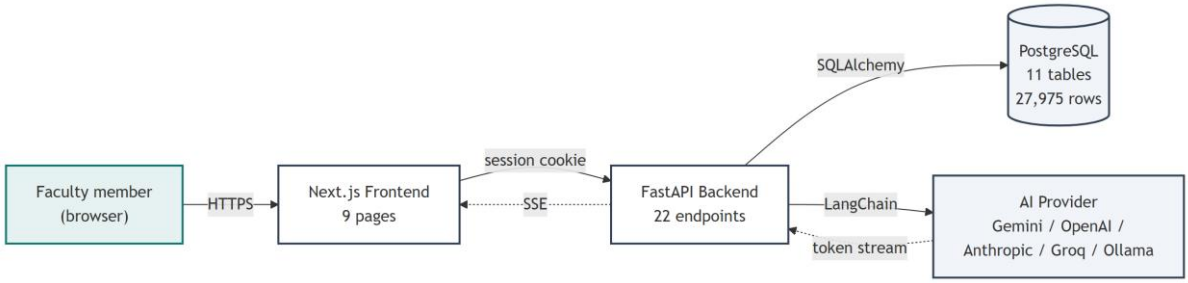


Figure 4.1: System context and external interfaces

4.2 Features and Scope

Module	Capability
Faculty Dashboard	Subject count, students in scope, below-threshold totals, overdue work, attendance chart, upcoming deadlines
My Subjects	Assigned subjects with enrolment and aggregate attendance
Attendance Analytics	Per-student attendance with filters, sorting, search, pagination and authorised CSV export
Student Directory	Students within scope, with a detail view of permitted records
Academic Performance	Weighted scores, marks distribution, evaluation progress, threshold filtering
Assignments	Submission rollups distinguishing evaluated, submitted-unmarked and genuinely overdue work
Extracurricular	Aggregate participation by category and role; department-wide view for a coordinator only
AI Assistant	Natural-language questions answered from authorised database functions, streamed progressively
Settings	Provider selection, model, API key entry, connection test, and the teacher's resolved permissions

Table 4.1: Functional modules and their capabilities

Out of scope: the system is read-only with respect to student records. No function available to the model or the interface modifies attendance, marks or submissions. There is no student-facing portal; students are the subject of analysis, not users.

4.3 System Architecture

The architecture separates four concerns: presentation, authorisation, business logic and AI orchestration. Authorisation is a single module that both the REST routes and the AI functions call

into, so no second implementation can drift from the first. Calculations likewise live in a shared services layer, so a figure on the dashboard and the same figure quoted by the assistant cannot disagree.

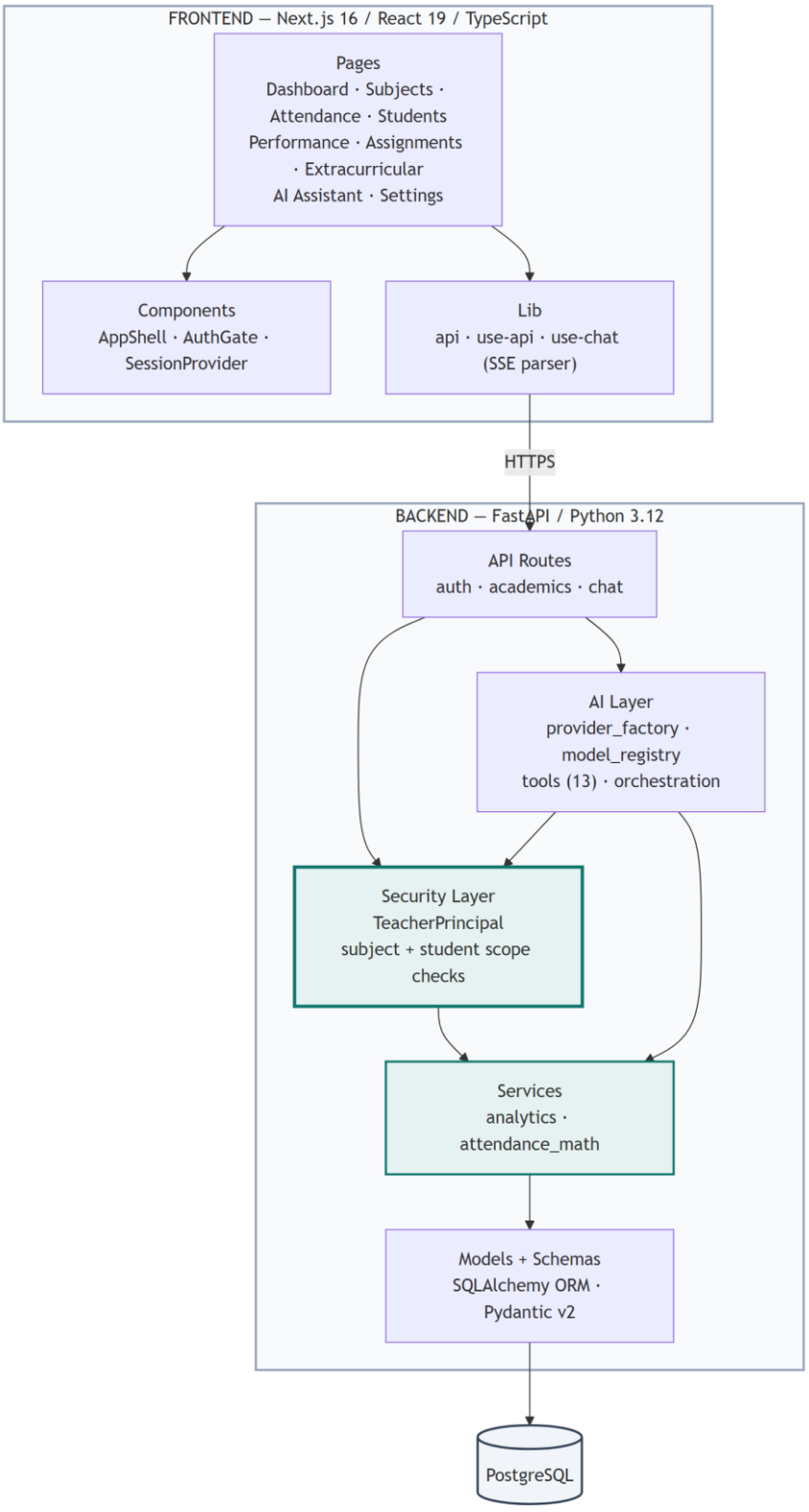


Figure 4.2: Component architecture and layer responsibilities

4.4 Database Design

Eleven normalised tables hold the records. The teacher_subjects table is the authoritative source for authorisation: it is the only place the subject-to-faculty mapping exists, and it is read fresh on every request so that a change in teaching assignment takes effect immediately rather than when a session expires.

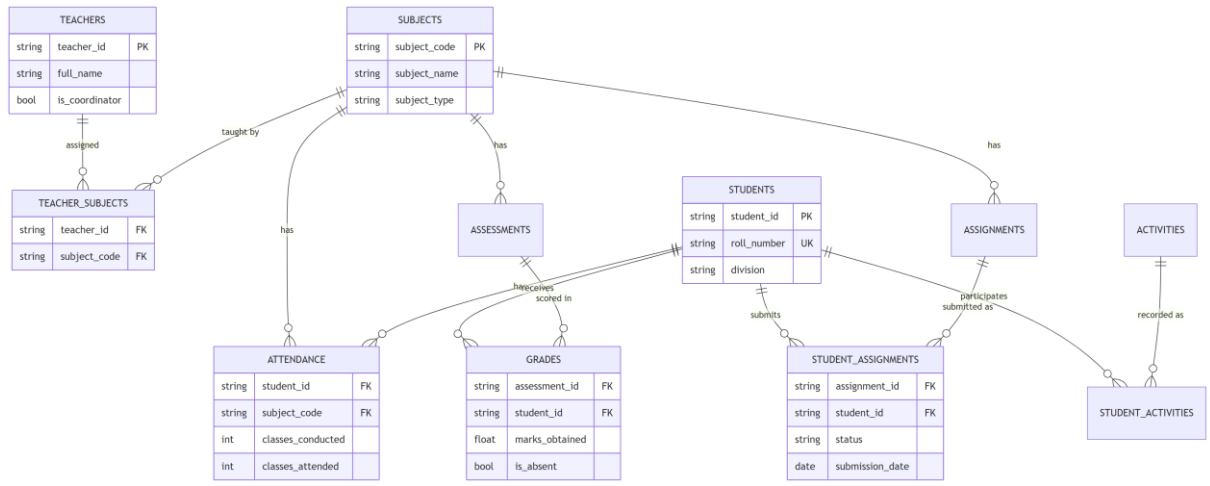


Figure 4.3: Entity-relationship diagram of the database schema

Integrity is enforced by the database itself, not only by application code, so that a programming error cannot write an impossible record:

- classes_conducted must be greater than zero, and classes_attended must lie between zero and classes_conducted.
- A student may have at most one attendance record per subject, and a teacher at most one assignment per subject.
- Submission status is restricted to pending, submitted or evaluated.
- Marks, where recorded, must be non-negative; a null mark denotes work not yet evaluated.

5. Methodology / Implementation

5.1 Tools and Technologies Used

Layer	Technology	Purpose
Frontend	Next.js 16, React 19, TypeScript	Application pages and routing with static type checking
Styling	Tailwind CSS 4	Utility-first styling with a restrained neutral palette
Charts	Recharts	Attendance and marks distribution visualisations
Backend	FastAPI, Python 3.12	Typed HTTP API with automatic OpenAPI documentation
Validation	Pydantic v2	Request and response schemas, including identifier patterns
ORM	SQLAlchemy 2.x	Parameterised database access and schema definition
Migrations	Alembic	Versioned schema evolution
Database	PostgreSQL 18 (Neon)	Managed relational storage
AI framework	LangChain 1.x	Uniform model interface, tool binding and streaming
Providers	Gemini, OpenAI, Anthropic, Groq, Ollama, and OpenAI-compatible services	Interchangeable language model backends
Testing	pytest, Vitest, Testing Library	Backend and frontend automated tests
Deployment	Vercel, GitHub Actions	Hosting and continuous integration

Table 5.1: Technology stack

5.2 Synthetic Dataset Generation

No real student information is used. A generator script seeded with a fixed value produces byte-identical output on every run, which allows the dataset to be regenerated and verified in continuous integration. Student email addresses use the .invalid top-level domain reserved by RFC 2606, so they can never resolve to a real address.

Attendance is generated from a latent diligence value per student, so a student's record correlates across subjects as it would in reality, while per-subject variation keeps the distribution realistic. The cohort is deliberately shaped to contain excellent, average, borderline and poor cases so that every analytic has data to exercise it.

Table	Rows	Notes
students	250	Divisions A, B and C; year 3, semester 6
subjects	12	Subject codes stored exactly as supplied, including suffixes
teachers	8	One per distinct faculty member; one department coordinator
teacher_subjects	12	Authoritative mapping used for authorisation
attendance	3,000	One record per student-subject pair

Table	Rows	Notes
assessments	60	Five weighted assessments per subject
grades	15,000	532 unevaluated, 312 absent, 217 genuine zeros
assignments	36	Three per subject with distinct due dates
student_assignments	9,000	5,489 evaluated, 1,759 submitted, 1,752 pending
extracurricular_activities	25	Ten categories from hackathons to volunteering
student_activities	572	200 of 250 students participate; awards are the exception
Total	27,975	All synthetic demonstration data

Table 5.2: Composition of the generated dataset

5.3 Authorisation Model

A teacher's identity comes only from a cryptographically signed session cookie. A teacher identifier appearing in a request body, a query string or a chat message is ignored entirely. On every request the backend reads that teacher's assigned subjects from the database and constructs a principal object carrying the resolved scope. Each endpoint then validates the requested subject or student against that scope before any data is read.

A subject that does not exist and a subject belonging to another teacher both return the same refusal, so the endpoint cannot be used to discover which subject codes are valid.

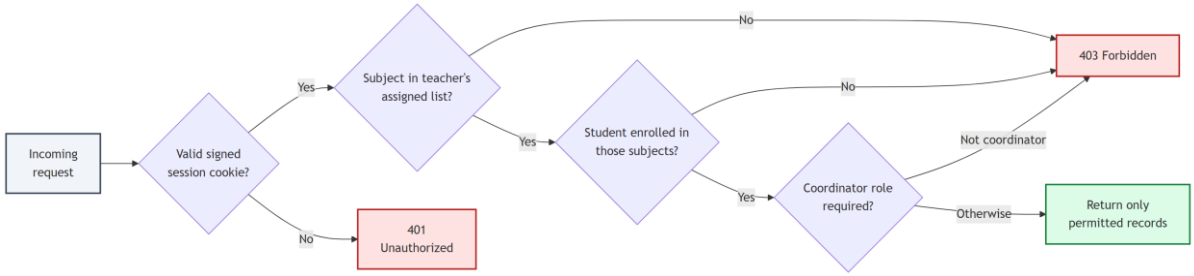


Figure 5.1: Authorisation decision sequence applied to every request

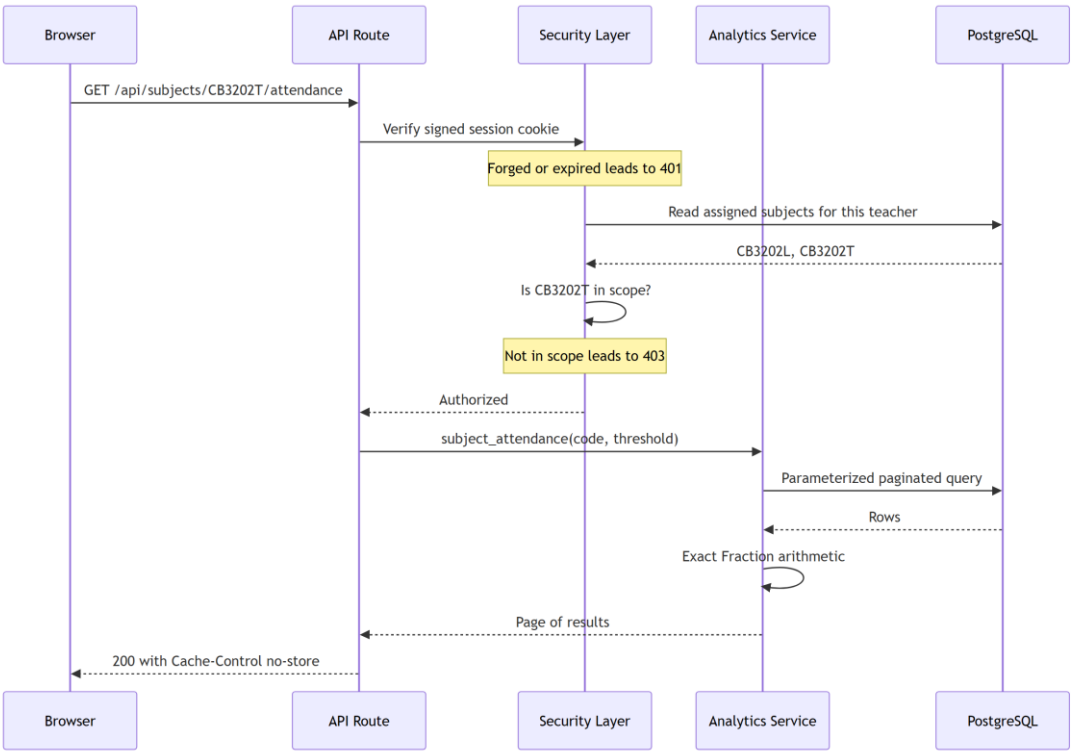


Figure 5.2: Request lifecycle from browser to database

5.4 Exact Attendance Arithmetic

All attendance calculations are implemented once, in a single module, using Python's exact rational Fraction type rather than floating-point numbers. This is not a theoretical nicety: thirty classes attended out of forty is exactly seventy-five percent, and with floating-point arithmetic such a student can be classified as below a seventy-five percent threshold depending on rounding. That would be a visible and unfair error in a report a teacher might act upon.

Attendance percentage is computed as classes attended divided by classes conducted, multiplied by one hundred.

The number of consecutive future classes a student must attend to reach a target percentage T, having attended A of C classes, is the ceiling of (T×C − A×100) divided by (100 − T), floored at zero. A target of one hundred percent after any absence is mathematically unreachable, and the implementation raises an error rather than returning a misleading figure.

Two different averages are reported and both are labelled in the interface. The pooled average divides total classes attended by total classes conducted, weighting every class equally. The mean student average takes the unweighted mean of each student's own percentage, weighting every student equally. These differ whenever subjects have different class counts, so presenting one unlabelled figure would misrepresent the cohort.

5.5 AI Orchestration

The language model is given thirteen functions and no database access. There is no SQL-executing function and no function that writes data. Each is bound to the authenticated teacher and re-checks scope against the database before reading, so a subject code or student identifier that the model invents simply fails authorisation and returns a refusal the model must relay.

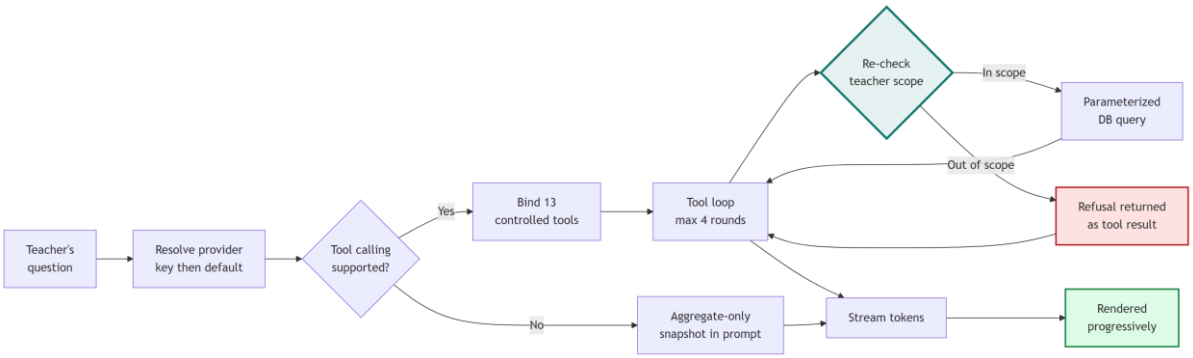


Figure 5.3: AI request flow with per-function authorisation

Category	Functions
Scope discovery	get_teacher_subjects, find_students
Attendance	get_subject_attendance_summary, get_low_attendance_students, get_student_attendance, get_classes_needed_for_target
Academic performance	get_subject_performance_summary, get_students_below_marks_threshold, get_student_academic_summary
Assignments	get_assignment_submission_summary, get_student_assignment_status
Extracurricular	get_extracurricular_summary, get_student_extracurricular

Table 5.3: The thirteen access-controlled functions available to the model

Results are capped at forty rows and every function reports the full matching total separately from the number of rows displayed, together with an explicit truncation flag. Function results also carry the definitions they depend upon — the threshold applied, what counts as overdue, how unevaluated work was treated — so that the model restates them accurately rather than inferring them.

Models that do not support function calling, such as certain reasoning-oriented models, are detected from a capability registry. For these, the backend places an aggregate-only snapshot in the prompt; no individual student record is ever written into prompt text, and the assistant states that per-student answers require a function-calling model.

5.6 Streaming and Credential Handling

Responses are delivered using Server-Sent Events. Tokens are forwarded to the browser as the provider emits them, with no buffering of the complete response and no simulated typing animation. The interface supports stopping generation mid-stream, retrying a failed request, copying a response and clearing the conversation. When a provider request fails, the failure is reported with a retry option; it is never replaced with invented text.

A user-supplied API key is held only in browser memory. It is never written to local storage, a cookie, a URL or the database, and is transmitted to the backend only with the request that needs it, used to construct a client for that single call, and then discarded. A redaction filter strips credential-shaped strings from every log record, and provider exceptions are rewritten into actionable messages before display, because provider SDKs routinely echo the authorisation header into their error text.

Custom provider endpoint URLs are validated before use. Each hostname is resolved and all returned addresses checked, rejecting private, loopback, link-local and reserved ranges. This prevents the endpoint field being used to probe internal infrastructure, including cloud metadata services. Loopback addresses are permitted only for locally hosted models, which is a documented development configuration.

6. Results and Discussion

6.1 Outputs Achieved

The system is complete and deployed. All nine interface modules render live data from the database, the assistant answers questions from authorised functions, and authorisation behaves correctly when probed directly over HTTP as well as through the interface.

Component	Delivered
Database	11 tables, 27,975 synthetic records, constraints enforced at database level
Backend API	22 typed endpoints with authorisation on every data route
Frontend	9 responsive pages with loading, empty and error states
AI layer	13 access-controlled functions across 9 supported model providers
Streaming	Server-Sent Events with stop, retry, copy and clear
Tests	208 automated tests (182 backend, 26 frontend), all passing
Code	Approximately 11,200 lines across backend and frontend
Deployment	Frontend and backend deployed; continuous integration passing

Table 6.1: Delivered components

6.2 Verification and Test Coverage

Testing is organised around the properties that matter rather than line coverage. Backend tests run against an in-memory database built from the same schema definitions, and frontend tests stub the network layer with a controllable stream, so the entire suite runs without a live database or any API credential.

Suite	Tests	Properties verified
Dataset integrity	39	Exactly 12 subjects, faculty names preserved, valid references, attendance within bounds, all evaluation states present, byte-identical regeneration
Attendance arithmetic	25	Percentages, exact threshold boundaries, the classes-needed formula checked against its own definition, invalid inputs rejected
Business logic	28	Zero distinguished from missing, absent from unevaluated, overdue from unmarked, pooled from mean averages
Authorisation	30	Forged and expired sessions rejected, cross-teacher access denied, coordinator role gated, export scope and filters honoured
AI layer	60	Provider configuration, endpoint validation, credential redaction, per-function authorisation, progressive streaming, provider failure, injection resistance
Frontend	26	Stream parsing including split frames, stop and retry, error handling, missing values rendered as a dash rather than zero

Table 6.2: Automated test coverage by suite

6.3 A Defect Found by Probing the Deployed Assistant

After deployment the assistant was probed with deliberately difficult questions rather than assumed correct. Asked an open-ended question about which students needed attention, it replied

that a total of fifteen students were below the attendance threshold. The correct figure was 154 student-subject records across 96 distinct students.

The cause was not the wording of the prompt but the design of the function. It returned only the number of rows in its capped sample, with no field expressing the full total. The model had no means of knowing the population size and reported what it could observe. No change to prompt wording could have corrected this, because the required information was absent from the data the model received.

The functions were changed to report the total matching records, the number of distinct students affected, the number of rows displayed and an explicit truncation flag. The system prompt gained a counting rule naming the specific fields to cite and forbidding any count inferred from the length of a list. The same question now returns the correct figure, and the distinction between records and people is stated explicitly, which matters because a student below the threshold in both a theory and a laboratory paper is two records but one person.

	Before the fix	**After the fix**
Assistant's answer	“A total of 15 students are recorded below the threshold”	“154 student-subject records across 96 distinct students below the 75.0% threshold”
Correctness	Incorrect by an order of magnitude	Matches the database exactly
Truncation	Not disclosed	Stated alongside the total

Table 6.3: Effect of the counting fix on assistant output

This episode is reported because it illustrates the project's central thesis. The safeguard that worked was not an instruction but a change to the data the model received. A plausible, confident and entirely wrong number is the characteristic failure mode of language models over structured data, and it is prevented by design rather than by exhortation.

6.4 Authorisation Verification

Authorisation was verified against the deployed system rather than only in tests. Signed in as a teacher assigned two Machine Learning papers, the following results were observed:

Request	Expected	Observed
Own assigned subject	200 OK	200 OK
Another teacher's subject	403 Forbidden	403 Forbidden
Student outside teaching scope	403 Forbidden	403 Forbidden
No session cookie	401 Unauthorized	401 Unauthorized
Forged session cookie	401 Unauthorized	401 Unauthorized
Department report as non-coordinator	403 Forbidden	403 Forbidden
Department report as coordinator	200 OK	200 OK
Chat message with a forged system role	422 Unprocessable	422 Unprocessable

Table 6.4: Authorisation behaviour observed on the deployed system

The assistant was additionally instructed to disregard its restrictions and disclose another teacher's subject. It refused, correctly naming the authenticated identity and the actual permitted scope. The refusal is reassuring but not the protection itself: the underlying function would have denied the request regardless of the model's decision.

6.5 Comparison with Existing Approaches

Property	Text-to-SQL	RAG	This system
Numerical accuracy	Depends on generated query	Approximate retrieval	Exact, computed in Python
Access control	Expressed in prompt text	Difficult to express per-user	Enforced in code per function
Auditability	Queries vary per request	Retrieval is opaque	Finite, reviewable function set
Failure mode	Silently wrong query	Plausible wrong context	Explicit refusal or error
Added infrastructure	None	Vector store and embeddings	None

Table 6.5: Comparison with alternative architectures

No vector database, retrieval pipeline or agent framework was introduced. The data is relational, and the questions teachers ask have exact answers reachable by indexed queries. Adding retrieval infrastructure would have increased complexity and latency while reducing accuracy, and would have made per-teacher authorisation substantially harder to express.

7. Conclusion and Future Scope

7.1 Summary of Achievements

A complete teacher-facing student information assistant was designed, implemented, tested and deployed. The system demonstrates that a language model can provide a natural-language interface to institutional records without being trusted with either the calculations or the access control.

- A reproducible synthetic dataset of 27,975 records whose regeneration is verified byte-for-byte in continuous integration.
- Authorisation enforced in a single server-side module, re-checked inside every function the model can call.
- Exact rational arithmetic in one shared location, so a student at the threshold is never misclassified.
- Thirteen access-controlled functions with no SQL generation and no write capability.
- Genuine progressive streaming across nine interchangeable model providers, with honest failure reporting.
- 208 automated tests covering data integrity, business logic, authorisation and AI behaviour.
- A deployed system whose authorisation behaviour was verified by direct probing, not assumed.

7.2 Limitations

The limitations below are stated plainly because a report claiming none would be less useful than one that identifies where the work stops.

- Authentication is a demonstration selector. Choosing a name issues a session with no credential check. The authorisation model that follows is real, but the identity establishment is not.
- Serverless deployment constrains streaming. The hosting platform imposes a sixty-second function limit, so a long response can be truncated. Container-based deployment configurations are included and remove this constraint without code changes.
- Rate limiting is per-instance. Counters are held in process memory, so limits weaken across multiple instances. A shared store such as Redis would be required in production.
- Provider integrations are verified against one provider. Streaming, function calling and error handling are tested comprehensively with mocks, and verified end to end against Google Gemini. The remaining eight providers are implemented but not confirmed against their live APIs.
- No audit trail exists. Access to records is not logged, which would be mandatory before any deployment involving real student information.

7.3 Future Scope

- Replace the demonstration sign-in with institutional single sign-on and multi-factor authentication.
- Add an append-only audit log recording who accessed which record and when.

- Introduce historical snapshots so period-over-period comparisons become answerable rather than correctly refused.
- Extend the role model to heads of department, mentors and examination staff with distinct scopes.
- Add scheduled report generation and configurable alerts when a student crosses a threshold.
- Evaluate locally hosted models so that student data need not leave institutional infrastructure.
- Conduct accessibility auditing against WCAG 2.1 AA and a formal security review.

7.4 Reflection

The most instructive outcome of this project was not a feature but a defect. The assistant confidently reported fifteen students where the true figure was 154, and the fault lay in a function that returned a sample without stating the population. It was corrected by changing what the model was given, not by instructing it more firmly. The general lesson is that when a language model is placed over structured data, correctness is a property of the interface the model is handed, and safety is a property of the code that executes beneath it. Neither can be delegated to the prompt.

8. References

[1] FastAPI Documentation. *Sebastián Ramírez. https://fastapi.tiangolo.com/*

[2] Next.js Documentation (App Router). *Vercel. https://nextjs.org/docs*

[3] LangChain Documentation. *LangChain Inc. https://python.langchain.com/docs/*

[4] SQLAlchemy 2.0 Documentation. *SQLAlchemy Project. https://docs.sqlalchemy.org/*

[5] Pydantic Documentation. *Pydantic Services. https://docs.pydantic.dev/*

[6] PostgreSQL 18 Documentation. *PostgreSQL Global Development Group. https://www.postgresql.org/docs/*

[7] Gemini API Documentation. *Google. https://ai.google.dev/gemini-api/docs*

[8] Server-Sent Events, HTML Living Standard. *WHATWG. https://html.spec.whatwg.org/multipage/server-sent-events.html*

[9] RFC 2606: Reserved Top Level DNS Names. *D. Eastlake and A. Panitz, IETF, 1999. https://www.rfc-editor.org/rfc/rfc2606*

[10] OWASP Top 10 for Large Language Model Applications. *OWASP Foundation. https://owasp.org/www-project-top-10-for-large-language-model-applications/*

[11] Rajpurkar, P. et al., “Know What You Don’t Know: Unanswerable Questions for SQuAD”. *Proceedings of ACL, 2018.*

[12] Ji, Z. et al., “Survey of Hallucination in Natural Language Generation”. *ACM Computing Surveys, vol. 55, no. 12, 2023.*

[13] Recharts Documentation. *Recharts Group. https://recharts.org/*

[14] Tailwind CSS Documentation. *Tailwind Labs. https://tailwindcss.com/docs*

Project Repository and Live Deployment

Source code: https://github.com/prasadbhalerao1/Student-Enquiry-Chatbot

Live application: https://frontend-kappa-puce-90.vercel.app

API documentation: https://csbs-faculty-api.vercel.app/docs

All student records referenced in this report are synthetic demonstration data generated for this project. They describe no real individual and are not institutional records.